

Achilles LoRA Eğitimi — Detaylı Anlatım

Sürüm: v1.1 · 2026-06-16

Sürüm Geçmişi

Sürüm	Tarih	Değişiklik
v1.0	2026-06-16	İlk kapsamlı sürüm
v1.1	2026-06-16	Denetçi düzeltmeleri: <code>check_flags</code> fiili davranışı, <code>cloud_notebook.py</code> net durumu, <code>chunk_size=1200</code> kesinleştirildi, <code>classify_curriculum</code> imzası iki-parametrelili olarak düzeltildi

Not: Yeni eğitim geliştirmesinde sürüm numarası artırılır ve değişiklik buraya eklenir.

Amaç ve Kapsam

Bu doküman, Achilles araştırma sisteminin **LoRA (Low-Rank Adaptation) eğitim hattını** uçtan uca anlatır. Hat, akademik PDF literatüründen başlayıp, denetlenmiş veri üretimine, kalite kapılarına, yerel/bulut eğitimine, değerlendirmeye ve nihayet üretime terfi eden bir adapter'a kadar uzanır.

Kapsam:

- Veri üretimi** — iki bağımsız dataset builder (bilgi kartı + sentetik QA), RAFT/grounding, reasoning-chain verisi, birleştirme (unified), yakın-tekrar (near-duplicate) dedup ve müfredat seviyeleri (0–4).
- Kalite kapıları** — Gate 0–8, güvenlik tarayıcısı (BLOCKER) ve kontrol düzlemi (control plane) durum makinesi.
- Eğitim backend'leri** — MLX vs PEFT platform tespiti, dry-run vs `--run`, detached (tek-tık) süreç.
- LoRA profilleri** — `lora_profiles.yaml` ön-ayarları.
- Değerlendirme** — `adapter_eval` ile gerçek PEFT yükleme, base-vs-adapter yan-yanaya kıyas, v5 regresyon dersi, negasyon-kör uyarısı.
- Adapter registry yaşam döngüsü** — candidate → ... → production, kullanıcı onayı zorunluluğu.
- Bulut + aşamalı eğitim** — Kaggle/Colab, Stage 1/2, GGUF + Modelfile → Ollama.
- Loop'lar ve otomasyon** — LoRA ile ilgili tüm döngüler.

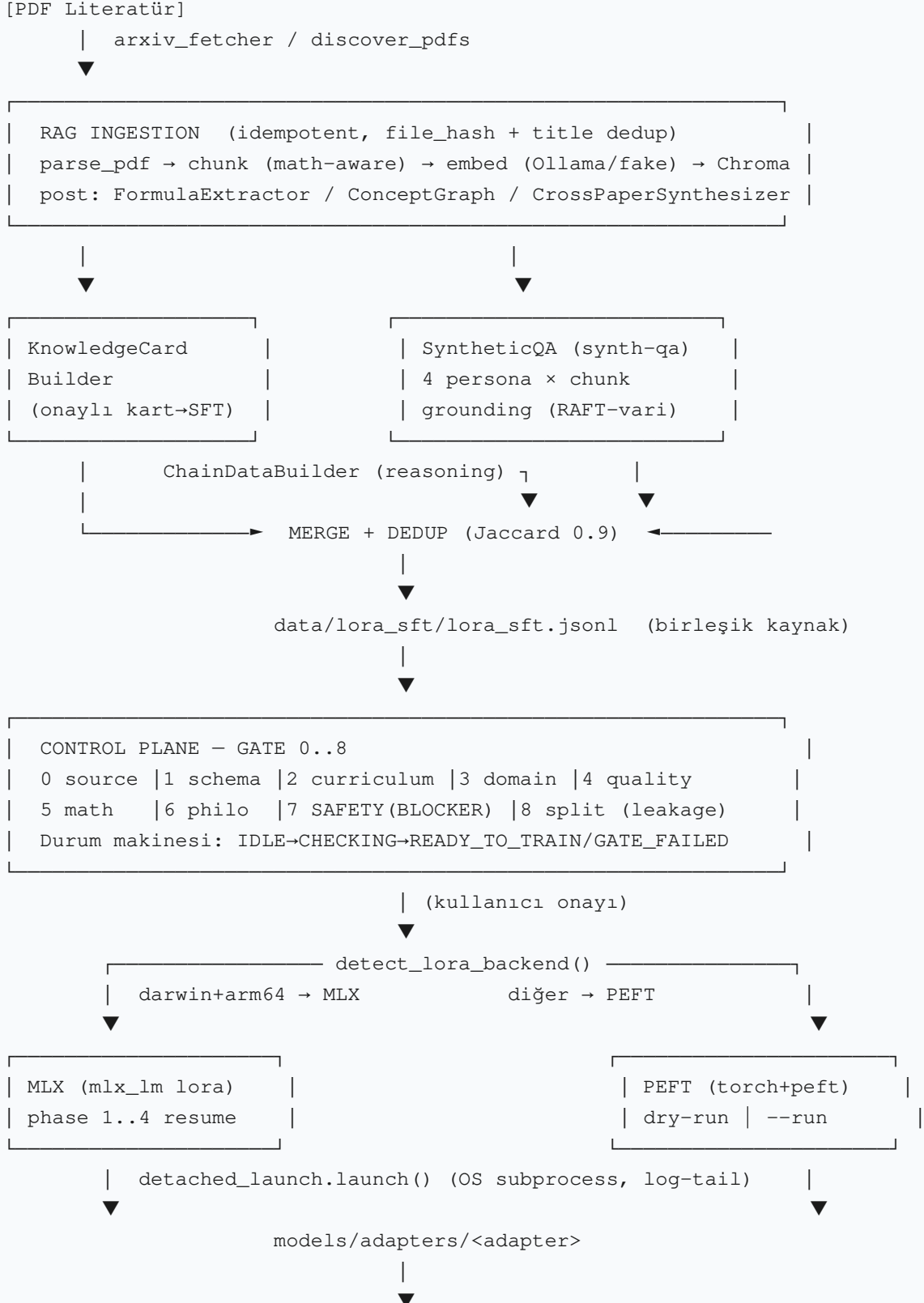
Bu doküman, CLAUDE.md kurallarına bağlıdır: kodda bulunamayan özellikler açıkça "kodda bulunamadı" olarak işaretlenir; hiçbir bileşen test edilmeden "çalışıyor" denmez; iddialar mümkün olduğunca `dosya:satır` ile desteklenir.

Önemli uyarı (durum): `storage/auto_lora_state.json` örneğinde sistemin son durumu `train_failed` ("Eğitim COMPLETED olmadı"), Gate sonucu ise `passed`

görülmektedir. Yani kalite kapıları geçilmiş ancak eğitim adımı tamamlanamamıştır. Doküman, hattın **tasarımını** anlatır; hattın baştan sona başarıyla koştuğu **iddia** edilmez.

Mimari Genel Bakış

ACHILLES LoRA EĞİTİM HATTI



```
adapter_eval - BASE vs ADAPTER (gerçek PEFT inference)
red-flag + degenerasyon → accept/reject/inconclusive
```

| EVAL_PASSED



```
ADAPTER REGISTRY (JSONL durum makinesi)
candidate → smoke_passed → eval_passed → approved → PRODUCTION
promote() ← user_approved=True ZORUNLU
```

—— ALTERNATİF YOL: ≥1000 örnek → BULUT (Stage 2) ——

lora-cloud-prep → Unsloth notebook (Kaggle/Colab T4)

→ GGUF Q4_K_M + Modelfile → ollama create → eval gate → promote

Aşama Aşama Yöntemler

Aşama 0 — RAG Ingestion (veri kaynağının hazırlanması)

LoRA verisi, doğrudan PDF korpusundan türeyen yapılandırılmış bilgiye dayanır. Bu nedenle ingestion, eğitim hattının "sıfır" aşamasıdır.

- **NE:** PDF → metin → matematik-korumalı chunk → embedding → Chroma + SQLite; ardından formül/kavram/sentez post-processing.
- **NASIL:** `PaperIndexer.ingest_one()` idempotent çalışır: önce `file_hash` (aynı bayt → skip), sonra normalize edilmiş başlık (`find_paper_by_title`) ile çift-kayıt engellenir. Chunking, `$$...$$`, `\[...]`, `\begin{equation}` ve `%12+` matematiksel karakter oranı olan paragrafları **bölmez** (`_is_math_heavy`).
- **DOSYA:** `app/memory/paper_indexer.py:63-215`, `app/ingestion/chunker.py:23-156`, `app/ingestion/pdf_parser.py:56-73`, `app/ingestion/arxiv_fetcher.py:48-164`.
- **MODEL/KÜTÜPHANE:** PyMuPDF/pypdf (PDF), Ollama `nomic-embed-text` (embedding; offline'da deterministik SHA256 fake fallback), ChromaDB (HNSW cosine), SQLAlchemy 2.0 (SQLite).
- **PARAMETRELER:** `chunk_size=1200` (kesin varsayılan, `app/config/settings.py:77`), `chunk_overlap=200` (`settings.py:78`); chunker bu değerleri `settings.chunk_size`'tan alır (`app/ingestion/chunker.py:85`). `rag_contextual_embed` (varsayılan kapalı). Not: `synthetic_qa_builder` içindeki `_MAX_PASSAGE_CHARS=1800`, prompt'a gömülen pasaj üst sınırıdır; `chunk_size` ile karıştırılmamalıdır.

Doğrudan LoRA verisi üretmediği için bu aşama eğitim hattının ön-koşuludur; detaylı RAG dokümanında genişler. Burada yalnızca formül/kavram/sentez çıktısının `training_examples` tablosuna yazıldığını not ederiz (`CrossPaperSynthesizer.synthesize_all()`).

Aşama 1 — Veri Üretimi (iki dataset builder)

1a. Bilgi Kartı Dataset Builder

- **NE:** SQLite `knowledge_cards` tablosundan onaylı kartları (`review_status=approved AND lora_eligible=1`) OpenAI-uyumlu `messages` SFT örneğine çevirir.
- **NASIL:** `card_to_lora_example()` filtreyi uygular (onaysız/uygunsuz → None); `_build_answer()` özet + ana iddia + trading geçerliliği + hipotezler + formülleri birleştirir. Metadata: `source_id` (dedup/split için), `difficulty` [0,1], `stage` (`lora_phase_1..4`).
- **DOSYA:** `app/lora/dataset_builder.py:34-114`.
- **MODEL/KÜTÜPHANE:** Saf Python + SQLAlchemy; bu aşamada LLM yok (kart üretimi zaten yapılmış sayılır).
- **NOT:** Kartların kendisi `KnowledgeCardBuilder.build()` ile üretilir (LLM, temperature 0.1, Ollama JSON modu, retry boş `main_claim`'de; `app/brain/knowledge_card_builder.py:138-180`). Güven seviyesi daima "draft" — insan onayı zorunlu.

1b. Sentetik QA Motoru (synth-qa) — RAFT-benzeri grounding

- **NE:** Makale chunk'larından LLM ile çeşitli, **grounded** (yalnız pasaja dayalı) Türkçe QA örnekleri üretir. Hedef: ~15–50 şablondan ~1000–2000 örneğe büyüme.
- **NASIL:** Her chunk bir **persona** alır (kantitatif araştırmacı / risk yöneticisi / backtester / şüpheli denetçi — `PERSONAS`, `synthetic_qa_builder.py:34-51`), rotasyonla. `build_for_chunk()` prompt kurar, deterministik seed (`eff_seed = seed + persona_index`), `temperature=0.4`, `fmt="json"`, `max_tokens=1600`. JSON parse → normalize → **grounding** üç-kapı:
 1. Cevaptaki anlamlı sayılar (`_SIG_NUM_RE = \d+[\.,]\d+|\d+%\d{2,}`) pasajda geçmeli;
 2. bilgi-fakir pasaj reddi;
 3. en az 1 anchor örtüşmesi. Ardından uzunluk filtresi (`min_answer_chars=60`).
- **DOSYA:** `app/brain/synthetic_qa_builder.py:111-138` (`_is_grounded`), :216–238 (`_build_prompt`), :241–317 (`build_for_chunk`), :319–347 (`build_for_paper`).
- **MODEL/KÜTÜPHANE:** Ollama `qwen3:4b` (enjekte edilebilir LLM → çevrimdışı test).
- **PARAMETRELER (CLI):** `synth-qa --per-chunk 5 --max-chunks 12 --max-papers 0 --append --seed 0`; toplu: `synth-qa-bulk --batch 5 --target 1000` (checkpoint'li, çökme-güvenli; tam batch implementasyonu kaynak bulgusunda "tam okunmadı" notuyla **kısmen doğrulandı**).

RAFT notu: Direktifte "RAFT" geçer. Kaynak bulgusu (`lora-dataset`) açıkça belirtir: **"RAFT seed" terimi kodda geçmez**; mevcut grounding mekanizması RAFT'ın (Retrieval-Augmented Fine-Tuning) "cevap yalnız verilen pasajdan kaynaklanmalı" ilkesinin uygulamasıdır, ancak ayrı bir RAFT modülü **kodda bulunamadı**. Repo kökünde `storage/_gen_raft_seed.py` adlı izlenmemiş bir dosya görülmektedir (git status); içeriği bu bulgulara yer almadığından işlevi **doğrulanmadı**.

1c. Chain Data Builder (reasoning zinciri)

- **NE:** Araştırma oturumlarını (`research_sessions`) "nasıl düşünmesi gerektiğini" öğreten örneklerle çevirir.
- **NASIL:** Prompt = soru + mevcut formüller + kavram grafiği; completion = Düşünce → Önerilen indikatör → Bileşenler → Giriş kuralları → Beklenen avantaj → Başarısızlık koşulları → Backtest (Sharpe/getiri/verdict) → Yansıma → İyileştirme.
- **DOSYA:** `app/research/chain_data_builder.py:96-174`. Çıktı: `data/training/`

1d. Unified Dataset Builder

- **NE:** Üç kaynağı (kartlar + mastery sınavları + tool-use seansları) tek MLX-formatlı (`{"prompt", "completion"}`) dosyada birleştirir.
- **NASIL:** `DatasetBuilder.collect()` + `MasterySFTBuilder.collect()` + `build_tool_use_dataset()` → `random.Random(seed).shuffle` → yaz.
- **DOSYA:** `app/training/unified_dataset.py:50-108`. Çıktı: `data/training/unified_sft.jsonl`.
- **NOT:** `MasterySFTBuilder` (`app/training/mastery_sft_builder.py`) ve `build_tool_use_dataset()` tanım dosyaları bu bulgularda **okunmadı** → iç davranış doğrulanmadı.

1e. Dedup + Müfredat + Bölme

- **Dedup:** `dedup_jsonl_lines()` — tam içerik SHA256 + yakın-tekrar Jaccard ≥ 0.9 (muhafazakâr, meşru farklılıklar korunur). İlk görülen tutulur. `app/brain/synthetic_qa_builder.py:427-468`.
- **Müfredat (0-4):** `classify_curriculum(card_json, difficulty)` — `LEVEL_0` (0.0-0.2) ... `LEVEL_4` (0.8-1.0). İmza iki konumsal parametre alır (`app/lora/curriculum.py:41`); `card_json` şu an kullanılmaz (`_ = card_json`, içerik-temelli genişletme için ayrılmıştır) ama **zorunludur** — tek argümanlı çağrı (`classify_curriculum(difficulty)`) hata verir. Sınıflandırma deterministik olarak yalnızca `difficulty`'ye dayanır. `app/lora/curriculum.py:41-59`.
- **Bölme (Gate 8 ön-fonksiyonu):** `split_dataset()` — `source_id` (`paper_id`) bazında grupta, `seed=42` shuffle, `train=0.8/valid=0.1/test=0.1`; **aynı makale tek bölmede** kalır. `check_leakage()` keşişimleri kontrol eder. `app/lora/dataset_splitter.py:40-103`.

Aşama 2 — Kalite Kapıları (Gate 0-8) ve Control Plane

- **NE:** Birleşik kaynağı sekiz sıralı kalite kapısından geçirir; temiz eğitim setini üretir ve raporlar. **Ağır eğitim başlatmaz.**
- **NASIL:** `LoRAControlPlane.run_audit()` (Gate 0-7) veya `run_full()` (Gate 0-8) → `PipelineReport`. Gate'ler saf fonksiyonlardır.

Gate	Ad	Kontrol	Tür
0	source	<code>review_status=approved</code> , ≥ 1 domain, <code>created_at</code>	BLOK
1	schema	<code>messages</code> list ≥ 3 ve sıra <code>[system,user,assistant]</code> , <code>metadata</code>	BLOK
2	curriculum	<code>difficulty</code> $\in [0,1]$	BLOK
3	domain	<code>classify_domains()</code> ≥ 1 domain	BLOK
4	quality	uzunluk < 50 / soru-cevap örtüşme ≥ 0.9 / SHA256 duplicate	BLOK (temiz çıktı)
5	math	<code>lookahead/survivorship</code> bayrakları (review) + aşırı-emin dil (BLOK)	BLOK+UYAR
6	philosophy	çelişki/korelasyon-nedensellik işareti	UYAR (blok değil)

7	safety	sır/PII/finansal yönlendirme	BLOCKER
8	split	train/valid/test + leakage	BLOK (sızıntıda)

- **DOSYA:** app/lora/gates.py:77-275, app/lora/control_plane.py:90-128, app/lora/quality_filter.py:62-121, app/lora/safety_scanner.py:14-75, app/lora/math_verifier.py:54-126, app/lora/domain_classifier.py:27-168.
- **GÜVENLİK TARAYICISI (Gate 7 / BLOCKER):** scan_for_secrets() regex setiyle: api_key (32+ alfanumerik), private_key (PEM başlığı), wallet_address, credential_assignment, email, phone (TR/int), national_id (11 hane); ayrıca FINANCIAL_DIRECTIVES ("şimdi al", "garanti kar", ...). **Tek ihlal tüm batch'i reddeder** — kısmi geçiş yoktur (safety_scanner.py:56-75, gates.py:244-259).
- **KONTROL DÜZLEMİ DURUM MAKİNESİ:** PipelineStage enum (auto_pipeline.py:27-38): IDLE → CHECKING → {GATE_FAILED | READY_TO_TRAIN} → TRAINING → {TRAIN_FAILED | EVALUATING} → {EVAL_SKIPPED | EVAL_FAILED | EVAL_PASSED} → PROMOTED. Durum storage/ auto_lora_state.json'da kalıcıdır.

Dürüstlük notu: Gate 4 duplicate izleme yalnız **oturum-içidir** (DB'de kalıcı duplicate log **kodda bulunamadı**). Gate 2 yalnızca difficulty aralığını kontrol eder; %60/%30/%10 müfredat pacing **kodda bulunamadı** (yalnız ajan tanımlarında metin olarak geçer).

Aşama 3 — Eğitim Backend'leri

3a. Platform Tespiti

- **NE:** Hangi backend? **NASIL:** detect_lora_backend() — sys.platform=="darwin" and platform.machine()=="arm64" → "mlx", aksi → "peft". **DOSYA:** app/training/backend.py:9-13. CLI'da --backend auto|mlx|peft (main.py:258).

3b. PEFT Eğitimi (Windows/Linux — bu projenin ana platformu)

- **NE:** torch + transformers + peft ile in-process LoRA eğitimi.
- **NASIL:** dry_run() varsayılan — --run yoksa eğitim **başlatılmaz** (sadece bağımlılık kontrolü + kurulum komutu). train() (--run ile): tokenizer/model yükle → LoraConfig uygula → tokenize (padding yok, dinamik collator) → Trainer.train() → loss eğrisi JSON.
- **DOSYA:** app/training/peft_lora_train.py:25-262, CLI app/main.py:244-321.
- **MODEL/KÜTÜPHANE:** Qwen/Qwen3-4B-Instruct-2507 (varsayılan base), torch/transformers/peft.
- **PARAMETRELER:** iterations=300, batch_size=1 (config) / 2 (CLI öneri, 8GB), lr=2e-4, lora_r=8, lora_alpha=16, dropout=0.05, max_seq_length=1024. target_modules = [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj] — **lm_head/embed YOK** (Qwen3 tied-embeddings; GGUF uyumu). dtype varsayılan fp32, ACHILLES_TRAIN_DTYPE=bf16 ile yarıya iner (AVX512-BF16 olmayan CPU'da emüle edilir → uyarı).

3c. MLX Eğitimi (macOS ARM)

- **NE:** mlx_lm lora alt-süreciyle eğitim; faz (1-4) resume desteği.

- **NASIL:** `build_command()` → `python -m mlx_lm lora --train --data <dir> --iters --batch-size --num-layers --adapter-path ...; from_phase>0 ise --resume-adapter-file. Bitince AdapterRegistry().register().`
- **DOSYA:** `app/training/mlx_lora_train.py:24-108`.
- **PARAMETRELER:** `iterations=300, batch_size=2` (8GB güvenli), `lr=1e-4, num_layers=8, save_every=0` (sona ertele → OOM önleme).

3d. Detached (tek-tık) Eğitim

- **NE:** Web/terminal kapansa da eğitimin sürmesi için DETACHED OS süreci.
- **NASIL:** `launch()` — adapter adı regex doğrulaması (`^[A-Za-z0-9_-]{1,64}$`), atomik kilit (`os.open(O_CREAT|O_EXCL)`, bayat kilit >120sn temizlenir), `ensure_train_split()` (clobber onarımı), `subprocess.Popen(..., creationflags=DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP | CREATE_NO_WINDOW)` Windows'ta / `start_new_session=True` POSIX'te. `stdout`→`logs/train-full.log`, `stderr`→`logs/train-full-err.log`. İlerleme tqdm satırından parse edilir (`(\d+)/(\d+)\s*<[...<eta)`, log <45dk yeni ise "canlı" sayılır.
- **DOSYA:** `app/training/detached_launch.py:51-359`.
- **HAZIRLIK:** `MIN_READY_EXAMPLES=200` (`data/lora_sft/lora_sft.jsonl` satır sayısı).
- **CLOBBER ONARIMI:** `ensure_train_split()` — birleşik kaynak doluysa HER ZAMAN `seed=42` ile yeniden böler (`valid_ratio=0.05`); kaynak boşsa mevcut `train.jsonl`'e dokunmaz.

Aşama 4 — LoRA Profilleri

- **NE:** Hazır eğitim ön-ayarları. **DOSYA:** `configs/lora/lora_profiles.yaml`.

Profil	r	alpha	dropout	epochs	target_modules	not
small_smoke_test	8	16	0.05	1	4 attn	<code>max_examples=20</code> pipeline doğrulama
standard_reasoning	16	32	0.05	2	7 (attn+MLP)	varsayılan
high_capacity_reasoning	32	64	0.05	3	7 (attn+MLP)	büyük veri

Aşama 5 — Değerlendirme (adapter_eval)

- **NE:** Eski `ModelEvaluator` base Ollama'yı ölçüyordu, adapter yüklemiyordu. Yeni `adapter_eval`, `transformers/PEFT` ile gerçek inference yapar ve base vs adapter yan-yana kıyaslar.
- **NASIL:** `_load_model(base, adapter_dir=None)` ile önce BASE, sonra `PeftModel.from_pretrained` ile ADAPTER; her ikisi `model.eval()`, greedy (`do_sample=False`, deterministik), `max_new_tokens=220`. Her cevapta:
 - **red-flag** (`check_flags`): fiilen yalnız iki deseni uygular — `guaranteed_profit` ve `ignores_costs` (ikincisi yalnız cevapta strateji/strategy geçiyorsa tetiklenir, `evaluate_model.py:84-87`) — artı soru-bazlı `must_avoid` token'ları. **NOT:** `success_without_test` deseni RED_FLAGS sözlüğünde tanımlıdır (`evaluate_model.py:32-34`) ancak `check_flags` veya başka hiçbir yerde kullanılmaz — ölü koddur (grep ile doğrulandı: tek geçtiği yer tanım satırı). Bu doküman onu aktif bir red-flag olarak listelemez.

- **degenerasyon** (`_is_degenerate`): benzersiz cümle $\leq$ yarısı $\rightarrow$ tekrar döngüsü. Puan:

`score = 1.0 - flags/n; verdict = accept | reject | inconclusive.`

- **DOSYA:** `app/training/adapter_eval.py:1-156`, `app/training/evaluate_model.py:28-91` (`check_flags :80-91`), CLI `app/main.py:339-374` (`lora-eval`).
- **MODEL/KÜTÜPHANE:** `transformers` (`AutoTokenizer`, `AutoModelForCausalLM`), `peft` (`PeftModel`), `torch` (`bfloat16`, `no_grad`).
- **EVAL SETİ:** `configs/eval/lora_eval_questions.yaml` (~50 soru, 6 alan, seviye 0–4).
YAML'deki `scoring_rubric/eval_thresholds` **kodda kullanılmaz** — yalnız `must_avoid token` kontrolü yapılır (**kodda bulunamadı:** rubric değerlendirme).

Aşama 6 — Adapter Registry Yaşam Döngüsü

- **NE:** Adapter durumunu izle; production'a geçiş **kullanıcı onayı** gerektirir.
- **NASIL:** `AdapterStatus` enum: `CANDIDATE` $\rightarrow$ `REJECTED` | `SMOKE_PASSED` $\rightarrow$ `EVAL_PASSED` $\rightarrow$ `APPROVED` $\rightarrow$ `PRODUCTION`. JSONL `append-only` + tam yeniden-yazım (`registry/adapters/registry.jsonl`). `promote(adapter_id, user_approved)` — `user_approved=False` ise hemen döner; mevcut production `APPROVED`'a indirilir (tek production garantisi), hedef `PRODUCTION` + `approved_by_user=True`.
- **DOSYA:** `app/lora/adapter_registry.py:20-160`, test `tests/test_lora_adapter_registry.py:37-63`.
- **NOT:** İkinci bir registry (`app/training/adapter_registry.py`, `SQLite` + `.meta.json` sidecar) vardır; kaynak bulgusu bunu **eski/kullanımda olmayan** olarak işaretler (yaşam döngüsü ve testler JSONL versiyonda).

Aşama 7 — Bulut + Aşamalı Eğitim (Stage 1/2)

- **NE:** Yerel CPU'da ≥ 1000 örnek eğitimi haftalar sürer ve overfit eder. Eşik aşıldığında eğitim ücretsiz bulut-GPU'ya (Kaggle T4x2 / Colab T4) taşınır.

Aspekt	Stage 1 (yerel CPU)	Stage 2 (bulut GPU)
İş	sentetik QA üret	gerçek LoRA fine-tune
Süre	sürekli	~30–90 dk
Çıktı	<code>synthetic_qa.jsonl</code>	GGUF Q4_K_M $\rightarrow$ Ollama

- **GATE 0 (nicelik):** `lora-readiness [--threshold 1000]` — sentetik satır + onaylı kart örneği toplamı $\geq$ eşik. `app/main.py:1715-1767`.
- **GATE 1–7 (audit):** `lora-audit` $\rightarrow$ `control_plane`. **KARAR:** ≥ 1000 örnek + audit geçti + kullanıcı onayı.
- **HAZIRLIK KOMUTU:** `lora-cloud-prep` — birleşik dataset + dedup $\rightarrow$ `lora_sft.jsonl`; Unsloth notebook (`build_stage2_notebook`, placeholder doldurma) $\rightarrow$ `notebooks/achilles_lora_stage2.ipynb`; Ollama Modelfile (`write_modelfile`) $\rightarrow$ ChatML TEMPLATE + `<|im_start|><|im_end|>` stop token'ları. `app/main.py:1770-1849`, `app/training/cloud_notebook.py:23-66`.
- **5 BİLİLEN HATA DÜZELTMESİ (notebook):** (1) 7 `target_module`, `lm_head/embed` yok (tied);

(2) {"messages": [...]} formatı; (3) apply_chat_template("qwen3-instruct"); (4) dinamik padding + train_on_responses_only (loss maskeleme); (5) save_pretrained_gguf("q4_k_m") + fallback (16-bit merge → llama.cpp).

- **KRİTİK EŞLEŞMELER:** base = Qwen/Qwen3-4B-Instruct-2507 (Ollama qwen3:4b-instruct-2507 ile birebir; çıplak qwen3:4b=2504 ile uyuşmaz); T4'te fp16 (bf16 DEĞİL — NaN); eğitim chat template ↔ Modelfile TEMPLATE birebir.
- **DOKÜMAN:** docs/PROTOKOL_ASAMALI_EGITIM.md, docs/PROTOKOL_BULUT_EGITIM.md.
- **NOT:** app/training/cloud_notebook.py **mevcuttur** (2379 bayt) ve build_stage2_notebook (:23) ile write_modelfile (:57) fonksiyonlarını içerir; app/training/peft_lora_train.py:280 içinde from app.training.cloud_notebook import build_stage2_notebook, write_modelfile ile import edilerek aktif olarak kullanılır (peft_lora_train.generate_colab_notebook tarafından çağrılır). Önceki "çelişki/belirsizlik" notu hatalıydı; tek ve net bir durum vardır.

Kullanılan Loop'lar ve Otomasyon

Loop / Mekanizma	Tetikleyici	Ne yapar
scripts/continuous-learning.sh	Manuel (CLI)	kart → onay → anlama → (3 turda bir) research+synth → synth-qa → rag-mastery; başta STOP_TRAINING ile eski eğitimi devralır
scripts/auto-chain.sh	Manuel	7 aşama: kart→onay→anlama→research→synth-paper→lora-dataset→ 24h eğitim döngüsü (train --run --backend peft --iterations 40)
scripts/mac-loop.sh	Manuel (macOS)	tur: kart→onay→synth-qa→lora-dataset→ MLX eğitim (300 iter); storage/train_status.json ile web rozeti güncelle
app/pipeline/auto_researcher.py	achilles auto-research	onaylı kart → hipotez sorusu → tool-use seans → reward skorla (DPO hazırlığı)
app/training/detached_launch.py	Web POST /api/training/launch/"EĞİTİME HAZIR" butonu	tek eğitim başlatma + atomik kilit + clobber onarımı + log-tail status
AutoLoRAPipeline.background_loop	auto_enabled=True (varsayılan False)	her aralıkta IDLE/GATE_FAILED ise check_and_prepare() (Gate 0–8)

ScheduleWakeup notu: Direktif "OS-süreci mi ScheduleWakeup mı" ayrımını ister.

Kaynak bulgularında **ScheduleWakeup** tabanlı bir nöbet mekanizması kodda **bulunamadı**; HANDOFF notu da "yeni seansta kendiliğinden devam ETMEZ" der. Tüm loop'lar ya manuel bash, ya web-tetikli detached subprocess, ya da web-içi asyncio'dur. Repo kökündeki `scripts/achilles-autostart.vbs / achilles-loop-autostart.vbs` (git status'ta izlenmemiş) Windows autostart için olabilir ancak içerikleri bu bulgularda yer almadığından **doğrulanmadı**.

Kararlar ve Dersler

- 1. Kaba yüzde yerine objektif sınav.** "Anlama %X" gibi öz-değerlendirme yerine, model SINAVLA (L1/L2 RAG sadakati; L3 sayısal `np.allclose`; L4 karşıolgu yön; L5 kompozisyon 3-kapı) ölçülür. LLM yoksa `skipped/no_data` — **sahte pass üretilmez** (`understanding_score.py:152-174`, `l3_application.py:89-100`). Bu, CLAUDE.md Kural 2'nin (test edilmeden "çalışıyor" deme) doğrudan uygulamasıdır.
- 2. v5 regresyon dersi.** v5 eğitimi tamamlandı ancak disiplinde GERİLEDİ. `reports/evals/adapter_eval_achilles_lora_v5_discipline_core.json`: adapter aynı ifadeyi 5 kez tekrarladı (`degenerate_repetition`), skorlar negatif (`base=-2.0`, `adapter=-1.0`), buna rağmen verdict `accept`. **Ders:** (a) degenerasyon cezası skoru daha ağır etkilemeli; (b) üretim öncesi manuel inceleme zorunlu; (c) eval harness'ın adapter'ı gerçekten yüklemesi şart — bu yüzden `adapter_eval` yazıldı.
- 3. Negasyon-kör (negation-blind) uyarısı.** `check_flags()` basit string eşleşmesidir (`token.lower() in answer.lower()`, `evaluate_model.py:88-89`) ve `guaranteed_profit/ignores_costs` desenleri de regex temellidir. "Kesinlikle **değil** kazanç" gibi olumsuzlamalar yanlış-pozitif bayrak tetikleyebilir. Negasyon-farkında kontrol **kodda bulunamadı** (`evaluate_model.py:82-91`).
- 4. Sürekli CPU-LoRA durduruldu, RAG-first + sentetik veri motoru.** 4B model CPU'da ~74–76 sn/adım; 15–50 örnek overfit. Karar: önce robust RAG + grounding'li sentetik QA ile ≥ 1000 örneğe ulaş, sonra bulut-GPU. (MEMORY: `rag-training-redesign`, `v5-adapter-regression`.)
- 5. eval/exec yasağı, whitelist AST.** Makaleden çıkan formüller `safe_eval` ile yalnız beyaz-listeli AST düğümleriyle yürütülür; L3/L4/L5 parse'ları yalnız JSON. (`safe_eval.py:44-92`.) CLAUDE.md Kural 5.
- 6. Determinizm her yerde.** Split `seed=42`, `synth-qa seed+persona_index`, `synthetic_closes(seed)`, greedy inference. CLAUDE.md Kural 6.
- 7. Production terfi yalnız kullanıcı onayıyla.** `promote(..., user_approved=True)` ve eval seti yoksa `EVAL_SKIPPED` → terfi bloklanır. CLAUDE.md Kural 8 + "Anayasa II/VI".
- 8. Clobber onarımı ve atomik kilit.** Detached eğitim sırasında kaynak güncellenirse `train/valid`'in 0'a düşmesi `ensure_train_split()` ile her başlatmada onarılır; çift-tık atomik kilitle engellenir.
- 9. Tied-embedding tuzağı.** Qwen3 tied-embeddings nedeniyle `lm_head/embed` delta'ları GGUF'ta sessizce atlanır → adapter bozulur. Çözüm: yalnız 7 attention+MLP modülü hedeflenir.

Dosya Referans Haritası

Dosya	Görev
app/lora/dataset_builder.py	Onaylı kart → LoRA SFT örneği (card_to_lora_example, _build_answer)
app/brain/ synthetic_qa_builder.py	Sentetik QA üretimi, grounding (RAFT-vari), dedup_jsonl_lines
app/brain/ knowledge_card_builder.py	PDF → yapılandırılmış bilgi kartı (LLM, JSON, draft)
app/research/ chain_data_builder.py	Araştırma oturumu → reasoning-chain eğitim verisi
app/training/unified_dataset.py	Kart + mastery + tool-use birleşik MLX dataset
app/lora/quality_filter.py	Gate 4: uzunluk/örtüşme/duplicate (SHA256)
app/lora/dataset_splitter.py	Gate 8: source-bazlı train/valid/test + leakage
app/lora/curriculum.py	Müfredat seviyeleri 0–4 (classify_curriculum(card_json, difficulty))
app/lora/gates.py	Gate 0–8 saf fonksiyonları
app/lora/safety_scanner.py	Gate 7 BLOCKER: sır/PII/finansal yönlendirme regex
app/lora/math_verifier.py	Gate 5: istatistik bias + aşırı-emin dil
app/lora/domain_classifier.py	Gate 0/3: anahtar-kelime domain sınıflandırma
app/lora/control_plane.py	Gate orkestrasyonu (run_audit/run_full), rapor
app/lora/auto_pipeline.py	Durum makinesi + background_loop + eval/promote akışı
app/training/backend.py	MLX vs PEFT platform tespiti
app/training/peft_lora_train.py	PEFT eğitimi (dry-run/--run), loss eğrisi, cloud_notebook import (:280)
app/training/mlx_lora_train.py	MLX eğitimi, faz resume
app/training/detached_launch.py	Detached subprocess başlatma + status + clobber onarımı
app/training/adapt_eval.py	Base-vs-adapt gerçek PEFT değerlendirme
app/training/evaluate_model.py	Red-flag kontrolleri (check_flags); success_without_test deseni tanımlı ama kullanılmaz (ölü kod)
app/lora/adapt_registry.py	JSONL yaşam döngüsü + promote (kullanıcı onayı)
app/training/ adapt_registry.py	(Eski) SQLite + sidecar registry

app/training/cloud_notebook.py	Stage 2 notebook + Modelfile üretimi (build_stage2_notebook, write_modelfile); peft_lora_train tarafından aktif kullanılır
configs/lora/lora_profiles.yaml	LoRA ön-ayar profilleri (r/alpha/epochs)
configs/eval/ lora_eval_questions.yaml	~50 soruluk disiplin eval seti
app/verification/exams/ understanding_score.py	L1–L5 sınav agregasyonu (objektif skor)
app/verification/exams/ safe_eval.py	Whitelist AST (eval/exec yok)
scripts/continuous-learning.sh	72h kart/anlama/synth-qa loop
scripts/auto-chain.sh	Tek-zincir + 24h eğitim loop
scripts/mac-loop.sh	macOS MLX eğitim loop
app/pipeline/auto_researcher.py	Kart → soru → tool-use → reward
app/main.py	CLI: train, lora-eval, lora-readiness, lora-audit, lora-cloud-prep, synth-qa (-bulk), auto-research
storage/auto_lora_state.json	Durum makinesi kalıcılığı

Bilinen Sınırlamalar

- Sistem son durumu train_failed.** storage/auto_lora_state.json örneğinde Gate geçilmiş ama eğitim tamamlanamamış. Hattın baştan-sona başarılı bir koşusu bu bulgularda **kanıtlanmadı**.
- RAFT modülü yok.** Grounding RAFT ilkesini uygular ama bağımsız "RAFT seed" mekanizması **kodda bulunamadı**; storage/_genRAFT_seed.py izlenmemiş/doğrulanmamış.
- Negasyon-kör red-flag.** Basit string/regex eşleşmesi olumsuzlamaları ayırt edemez.
- Degenerasyon cezası hafif.** v5 örneğinde tekrar döngüsüne rağmen accept; otomatik reject override **kodda bulunamadı**.
- Inference hata yönetimi yok.** adapter_eval'de model yükleme/inference başarısızlığında try/except **kodda bulunamadı** → crash.
- Eval rubric kullanılmaz.** YAML'deki scoring_rubric/eval_thresholds okunmaz; yalnız must_avoid token kontrolü.
- Cross-batch duplicate izleme yok.** Gate 4 yalnız oturum-içi hash; kalıcı DB log yok.
- Müfredat pacing / domain balance yok.** Gate 2/3 yalnız aralık/varlık kontrolü; %60/%30/%10 dağılım ve hedef domain dengesi **kodda bulunamadı**.
- Ölü red-flag deseni.** success_without_test deseni RED_FLAGS'te tanımlıdır (evaluate_model.py:32-34) ama check_flags veya başka hiçbir yerde çağrılmaz; etkin değildir. (Aktif desenler yalnız guaranteed_profit ve strateji-koşullu ignores_costs + must_avoid.)
- Çift registry.** JSONL (aktif) ve SQLite (eski) iki registry; karışıklık riski.
- MasterySFTBuilder / build_tool_use_dataset / ModelEvaluator / LoRAControlPlane.run_full iç detayı** bu bulgularda **tam okunmadı** → davranışları

doğrulanmadı.

12. **CPU'da eğitim pratik değil.** 4B model ~74–76 sn/adım; ≥ 1000 örnek için bulut-GPU zorunlu.

13. `auto_researcher` → **tool-use** zinciri DPO hazırlığı üretir; DPO eğitiminin kendisi (eğitim döngüsüne bağlanması) bu bulgularda **gösterilmedi**.

Sözlük

- **LoRA (Low-Rank Adaptation):** Tam modeli değil, eklenen düşük-ranklı (r) matrisleri eğiterek ucuz fine-tune.
- **SFT (Supervised Fine-Tuning):** Etiketli (soru→cevap / messages) örneklerle denetimli ince-ayar.
- **PEFT:** Parameter-Efficient Fine-Tuning kütüphanesi (torch+transformers); Windows/Linux yolu.
- **MLX:** Apple Silicon için ML çerçevesi; `mlx_lm lora` ile eğitim.
- **Grounding:** Cevabın yalnız verilen pasaja dayanması; uydurma metrik/sayı reddi.
- **RAFT:** Retrieval-Augmented Fine-Tuning; "yalnız bağlamdan cevapla" ilkesi (burada grounding olarak uygulanır).
- **Gate 0–8:** Sıralı kalite kapıları; bazıları BLOK, biri (7) mutlak BLOCKER, ikisi (5/6) UYARI.
- **BLOCKER:** Tek ihlalin tüm batch'i reddettiği kapı (Gate 7 güvenlik).
- **Control Plane:** Kapı orkestrasyonu + durum makinesi (`IDLE...PROMOTED`).
- **Detached süreç:** Web/terminal kapansa da süren bağımsız OS alt-süreci.
- **Clobber onarımı:** train/valid'in 0'a düşmesini her başlatmada kaynaktan yeniden bölerek önleme.
- **Dry-run:** Eğitimi başlatmadan bağımlılık/komut doğrulaması (`--run` olmadan varsayılan).
- **Degenerasyon:** Adapter'ın aynı cümleyi tekrarlaması (overfit/çöküş işareti).
- **Adapter registry:** Adapter durum + metadata defteri (JSONL); production'a terfi kullanıcı onaylı.
- **GGUF / Q4_K_M:** llama.cpp/Ollama model formatı / 4-bit k-quant kvantizasyon.
- **Modelfile:** Ollama model tanımı (FROM + TEMPLATE + SYSTEM + stop token).
- **Tied embeddings:** Giriş/çıkış embedding'lerinin paylaşıldığı mimari; GGUF'ta `lm_head delta'sı` atlanır.
- **Stage 1/2:** Yerel sentetik veri üretimi (1) → bulut-GPU LoRA eğitimi (2).
- **UnderstandingScore (L1–L5):** Anlamayı yüzdeyle değil sınavla kanıtlayan objektif skor.
- **source_id:** Örneğin kaynak makalesi; train/test sızıntısını önlemek için bölme anahtarı.
- **Ölü kod:** Tanımlı ama hiçbir çağrı yolunda kullanılmayan kod (örn. `success_without_test` deseni).